Übergabe und Arbeitsweise

Lernunterlage: Stand 31.10.2026, offene Punkte und wie man sauber weiterarbeitet

1 Projekt auf einen Blick

Schicht	Technik	Was sie macht
Frontend	React, TypeScript, Vite, CSS-Variablen	Oberfläche, Login, Status-LEDs, PWA
API	ASP.NET Core 10 Minimal API	REST-Endpunkte, ProblemDetails, OpenAPI/Scalar
Sicherheit	Identity, JWT, Refresh-Token, API-Schlüssel	Wer bist du? Was darfst du?
Daten	EF Core, SQLite, Migrationen	Räume, Geräte, Ereignisse, Benutzer, Tokens
Anbindungen	Tuya-Cloud (HMAC), Blink (inoffiziell)	Zustände und Online-Status lesen, nie schalten
Betrieb	systemd, Tailscale serve	Dienst mit Neustart, HTTPS-Link fürs iPhone

2 Der Arbeitsablauf (Git)

```
git switch main && git pull                 # aktuellen Stand holen
git switch -c feature/<thema>               # eigener Branch für die Aufgabe
# ... Code ändern ...
./pruefen.sh                                # Build, Migrationen, Tests, Frontend, Lint
git add <dateien> && git commit -m "Was und warum"
git push -u origin feature/<thema>          # dann auf GitHub einen Pull Request öffnen
# Review lesen, PR mergen (Merge-Commit), lokal: git switch main && git pull
```

- **Branch:** main bleibt immer lauffähig. Neue Arbeit passiert daneben.
- **Pull Request:** Der Schnappschuss zum Zeitpunkt des Mergens zählt. Später gepushte Commits landen nicht automatisch in main (das ist bei PR #5 passiert und wurde mit PR #6 nachgeholt).
- **Kleine Commits:** Ein Commit, eine Idee. Die Nachricht sagt, was und warum.
- **Ehrlichkeit:** Was nicht gebaut oder getestet wurde, steht in der Commit-Nachricht.

3 Was am 31.10. dazukam

Pull Request	Inhalt
#15 Stand geprüft	pruefen.sh, Migration IdentityUmgestellt, Startskript nutzt die echte Datenbank, 103/103 Tests grün
#16 Betrieb, Status-LEDs, Unterricht	Dienst + Tailscale-HTTPS, GET /api/status mit LEDs im Dashboard, Blink-Login-Schutz, AddValidation, GetUserId(), OpenAPI-Tags, Übergabe-Dokument, vier Lern-PDFs

4 Offene Punkte

1. PR #16 mit `./pruefen.sh` prüfen und mergen.
2. Dauerbetrieb einrichten: `./dienst-einrichten.sh`, `./tailscale-https-einrichten.sh`, iPhone/iPad.
3. KI-Agent: API-Schlüssel und https-Adresse in n8n, dann Lesen schützen.
4. Weitere Gerätetypen (Temperatur, Watt, Akku) nach den echten Tuya-Rohwerten.
5. Blink-Seite mit echter Kameraliste.
6. TypedResults in allen Endpunkten (wie im Unterricht).
7. Endpunktfilter gegen leere GUIDs.
8. Variante B: Betrieb auf dem OVH-Server mit Docker.

5 Mit einer KI weiterarbeiten

Eine KI kennt dein Projekt nicht. Gib ihr deshalb immer denselben Einstieg: `docs/UEBERGABE.md` lesen lassen, Regeln nennen, eine klar begrenzte Aufgabe geben. Die fertige Vorlage steht in UEBERGABE.md, Abschnitt 5.

Gut	Schlecht
„Lies docs/UEBERGABE.md. Aufgabe: Punkt 5, Blink-Seite mit Kameraliste.“	„Mach mal die App fertig.“
„Vor dem Commit ./pruefen.sh, Ergebnis zeigen.“	„Passt schon, wird schon laufen.“
„Erkläre mir, warum du das so machst.“	Code übernehmen, ohne ihn zu verstehen

> **Merksatz:** Die KI schreibt Vorschläge. Ob sie stimmen, entscheiden der Compiler, die Tests und du. Erst wenn `./pruefen.sh` grün ist und du es erklären kannst, ist es fertig.

6 Alle Lernunterlagen

PDF	Thema
Lernunterlage-Authentifizierung.pdf	JWT, Refresh-Token, Rotation, Rollen, XSS/CSRF
Lernunterlage-Identity.pdf	UserManager, IdentityUser, Rollen, Lockout
Lernunterlage-API-Schluessel.pdf	Zugang für den KI-Agenten
Lernunterlage-Tuya-Geraete.pdf	Tuya-Cloud, Statuscodes, Abgleich
Lernunterlage-Pruefen-und-Migration.pdf	pruefen.sh, Exit-Codes, Migrationen
Lernunterlage-Betrieb.pdf	systemd, dotnet publish, Tailscale-HTTPS
Lernunterlage-Status-LEDs.pdf	/api/status, Cache, Context, CSS-LEDs
Lernunterlage-Validierung-und-Unterricht.pdf	AddValidation, GetUserId, OpenAPI, Vergleich mit dem Unterricht
Lernunterlage-Uebergabe.pdf	diese Unterlage